UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Sistemas Operativos

Ejercicio Sistemas de Archivos

Actividad Ayudantía 3

Presenta:
Rojo Peña Manuel Ianluck - 118005762

Profesor:
Javier León Cotonieto

Ayudante laboratorio:
Fernanda Garduño Ballesteros

Grupo: 7004, 2026-2

Fecha de entrega:
24 de abril, 2026

Ejercicio 4.

> *¿Es la llamada al sistema* `open` *en UNIX absolutamente esencial? ¿Cuáles serían las consecuencias de no tenerla?*

La llamada al sistema `open()` recibe una ruta de archivo como argumento y devuelve un entero conocido como **descriptor de archivo** (*file descriptor*, FD). A partir de ese momento, todas las operaciones de lectura, escritura y posicionamiento se realizan a través de dicho número entero, no a través de la ruta original [1]. Internamente, el sistema operativo resuelve la ruta una sola vez, verifica los permisos del proceso y crea una entrada en la *tabla de archivos abiertos* del kernel que registra, entre otros datos, la posición actual de lectura/escritura (*offset*).

Técnicamente, un sistema operativo *podría* funcionar sin `open`, en lugar de obtener un descriptor, cada llamada a `read` o `write` recibiría la ruta completa del archivo como parámetro. Sin embargo, esto transformaría el sistema en uno **sin estado** (*stateless*) respecto a los archivos abiertos, con consecuencias graves:

1. **Colapso en el rendimiento por resolución repetida de rutas.** La traducción de una ruta textual como `/usr/ast/courses/os/handout.t` hacia la dirección física en disco implica recorrer cada componente del directorio, leer su bloque correspondiente e interpretar su i-nodo [1]. Con `open`, este proceso ocurre una sola vez; sin él, debería repetirse en cada llamada a `read` o `write`, incrementando drásticamente la cantidad de operaciones de E/S y degradando el rendimiento del sistema.

2. **Verificaciones de seguridad redundantes.** UNIX comprueba si el proceso tiene permiso para acceder al archivo en el momento en que se ejecuta `open`. Si esta llamada no existiera, el kernel debería verificar los permisos del proceso en cada operación individual de lectura o escritura, multiplicando el costo computacional de las comprobaciones de control de acceso [2].

3. **Pérdida del apuntador de posición (*offset*).** El estado creado por `open` permite que el sistema operativo mantenga el *offset* en la tabla de archivos abiertos, recordando en qué byte se quedó la última lectura. Sin este estado persistente en el kernel, el proceso tendría que especificar explícitamente la posición de inicio en cada llamada, trasladando al programador la responsabilidad de llevar la cuenta exacta, lo cual es ineficiente y propenso a errores [1].

4. **Ruptura de la filosofía «todo es un archivo».** La abstracción más influyente de UNIX es permitir que dispositivos de hardware, tuberías (*pipes*) y conexiones de red (*sockets*) se manejen con la misma simplicidad que un archivo de texto [3]. Esta unificación depende de .ªbrir.ᵉˡ recurso y obtener un descriptor unificado. Sin `open`, sería imposible aplicar llamadas genéricas de lectura/escritura a recursos sin rutas de disco duras, destruyendo la modularidad del sistema.

Analogía ilustrativa. Eliminar `open` sería equivalente a un hotel que no entrega llaves, cada vez que un huésped desea acceder a su habitación, debe ir a recepción, presentar su documentación, esperar a que el personal verifique la reserva y ser escoltado hasta la puerta. La estancia funciona, pero la recepción colapsaría en minutos ante la demanda concurrente de múltiples huéspedes.

En conclusión, no es lógicamente indispensable para que una computadora funcione, pero es arquitectónicamente esencial para el modelo de UNIX. Aunque en teoría un sistema podría operar sin `open`, en la práctica su ausencia haría que el sistema operativo fuera significativamente más lento, menos seguro y radicalmente más difícil de programar. La llamada `open` es arquitectónicamente esencial para mantener la eficiencia y la elegancia del modelo de E/S en UNIX.

Ejercicio 17.

Para una clase determinada, los registros de los estudiantes se almacenan en un archivo. Los registros son accedidos y actualizados de forma aleatoria. Se asume que el registro de cada estudiante tiene un tamaño fijo. ¿Cuál de los tres esquemas de asignación (contigua, enlazada e indexada/tabla) sería el más apropiado?

Los tres esquemas de asignación

Antes de elegir el esquema más adecuado, conviene comprender las características fundamentales de cada uno [1, 2]:

1. **Asignación contigua.** Los bloques de un archivo se almacenan en posiciones consecutivas del disco. Para acceder al bloque n de un archivo que comienza en el bloque b, el sistema operativo calcula directamente $b + n$. Esto hace que el acceso aleatorio sea $O(1)$, conociendo el tamaño fijo del registro y su número de orden, se puede calcular la dirección exacta en disco sin necesidad de recorrer ninguna estructura auxiliar. Su principal debilidad es la **fragmentación externa** y la dificultad para expandir el archivo si no hay bloques contiguos adyacentes disponibles.

2. **Asignación enlazada.** Cada bloque contiene un apuntador al siguiente bloque de la cadena. El acceso es secuencial por naturaleza, para leer el registro número n, es necesario recorrer los $n - 1$ bloques anteriores, siguiendo cada apuntador. Esto lo hace *totalmente inadecuado para acceso aleatorio*, pues el tiempo de acceso en disco crece linealmente con la posición del registro ($O(n)$).

3. **Asignación indexada (i-nodo / FAT).** Se mantiene una estructura índice que contiene las direcciones de todos los bloques del archivo. Para acceder al bloque n, se consulta la entrada n del índice y se obtiene la dirección directamente, el acceso aleatorio es posible, pero requiere un **overhead de Entrada/Salida**: una lectura en disco para el bloque índice más una lectura para el bloque de datos. No hay fragmentación externa y el archivo puede expandirse fácilmente [1].

Comparativa resumida

Criterio	Contigua	Enlazada	Indexada
Acceso aleatorio eficiente	✓ (1 I/O)	× (n I/O)	✓ (2 I/O)
Sin fragmentación externa	×	✓	✓
Expansión fácil del archivo	×	✓	✓
Overhead de almacenamiento	Ninguno	Apuntadores/bloque	Tabla o i-nodo

Elección más apropiada: Asignación Contigua

Bajo las restricciones literales del enunciado, la **asignación contigua** es indiscutiblemente la más apropiada. Los argumentos técnicos son los siguientes:

- **Cálculo directo (Overhead cero):** Al tener registros de tamaño fijo, la dirección física de cualquier registro se calcula mediante aritmética simple: Dirección Base + (Número de Registro $\times$ Tamaño del Registro). El sistema puede saltar al bloque exacto y realizar la actualización en una sola operación de disco.
- **Desempeño sobre la asignación indexada:** Aunque la asignación indexada permite acceso aleatorio, introduce una penalización de rendimiento (*overhead*). Por cada actualización, el sistema tendría que leer primero el bloque índice y luego acceder al bloque de datos (dos operaciones de E/S). Dado que la matemática contigua resuelve el acceso directo por sí sola, el uso de un índice aquí es redundante y menos eficiente en tiempo.
- **Ineficiencia de la asignación enlazada:** La necesidad de .ªcceso y actualización aleatoria"descarta inmediatamente la asignación enlazada debido al costo prohibitivo de recorrer la cadena de bloques uno por uno para cada consulta.

Nota de diseño (consideración del mundo real):
La respuesta óptima asume que el conjunto de estudiantes es estático, como sugiere el problema ("se asumen de tamaño fijoz solo menciona .ªccedidos y actualizados"). Si en un escenario práctico el diseño requiriera que la base de datos creciera dinámicamente con nuevas inscripciones de estudiantes de forma constante, la **asignación indexada** se volvería la opción más robusta para evitar tener que reubicar el archivo entero en el disco debido a la falta de bloques contiguos [2]. No obstante, para acceso puro aleatorio en un espacio predefinido, la contigua no tiene rival en velocidad.

Ejercicio 24.

El espacio libre en disco puede rastrearse usando una lista libre o un mapa de bits. Las direcciones de disco requieren D bits. Para un disco con B bloques, de los cuales F están libres, establezca la condición bajo la cual la lista libre ocupa menos espacio que el mapa de bits. Para $D = 16$ bits, exprese su respuesta como un porcentaje del espacio en disco que debe estar libre.

Estructuras de rastreo de espacio libre

El sistema operativo necesita saber qué bloques del disco están disponibles para ser asignados. Existen dos estrategias clásicas [1, 2]:

1. **Mapa de bits (*bitmap*).** Se reserva un arreglo de B bits, uno por cada bloque del disco. Si el bit i vale 1, el bloque i está libre; si vale 0, está ocupado (o viceversa, según la convención del sistema). El mapa de bits ocupa exactamente:

   $$\text{Espacio}_{\text{bitmap}} = B \text{ bits}$$

   Su principal característica es que **su tamaño es estrictamente constante**, independientemente de la cantidad de bloques libres u ocupados en el disco.

2. **Lista libre (*free list*).** Se mantiene una lista (usualmente enlazada o en bloques agrupados) de las direcciones de los bloques libres. Cada entrada almacena la dirección de un bloque libre, lo que requiere D bits por entrada, con F bloques libres, el espacio total ocupado por los datos de la lista es:

   $$\text{Espacio}_{\text{lista}} = F \times D \text{ bits}$$

   A diferencia del mapa de bits, el tamaño de esta estructura es **dinámico**, es muy compacta cuando el disco está casi lleno (el valor de F es pequeño), pero crece linealmente a medida que se libera espacio.

Condición general

La lista libre ocupa menos espacio que el mapa de bits cuando el costo de almacenar las direcciones completas es menor que el costo de tener un bit por cada bloque del disco:

$$F \times D < B$$

Despejando la fracción de bloques libres (F/B):

$$\frac{F}{B} < \frac{1}{D}$$

Es decir, la lista libre es preferible en espacio siempre que la proporción de bloques libres sea menor a $\frac{1}{D}$.

Caso particular: $D = 16$ bits

Sustituyendo $D = 16$:

$$\frac{F}{B} < \frac{1}{16} = 0{,}0625$$

Conclusión. Con direcciones de 16 bits, la lista libre ocupa menos espacio que el mapa de bits únicamente cuando menos del 6,25 % del disco está libre. En cualquier otro caso (disco con más del 6,25 % de espacio libre), el mapa de bits es más eficiente en almacenamiento. En la práctica, los usuarios rara vez operan sus discos a más del 93 % de su capacidad de forma sostenida, lo que explica por qué el mapa de bits (o variantes estructuradas en árboles) es la estructura preferida en los sistemas de archivos modernos [1].

Interpretación intuitiva

Cada entrada en la lista libre "cuesta" 16 bits, mientras que en el mapa de bits cada bloque cuesta exactamente 1 bit. La lista libre solo es ventajosa cuando hay tan pocos bloques libres que el ahorro de no representar los bloques ocupados compensa el costo inflado (16 veces mayor) de registrar explícitamente cada bloque libre restante.

Ejercicio 43.

¿Cuántas operaciones de disco son necesarias para obtener el i-nodo de un archivo con la ruta `/usr/ast/courses/os/handout.t`? Se asume que el i-nodo del directorio raíz está en memoria, pero ningún otro elemento de la ruta lo está. También se asume que todos los directorios caben en un solo bloque de disco.

Contexto: resolución de rutas e i-nodos

En UNIX, cada archivo y directorio está descrito por una estructura llamada **i-nodo** (*index node*), que contiene los metadatos del archivo (permisos, tamaño, marcas de tiempo) y las direcciones físicas de sus bloques de datos [1]. Para localizar un archivo a partir de su ruta absoluta, el sistema operativo debe **descender el árbol de directorios** nivel a nivel. En cada nivel, realiza dos operaciones fundamentales de Entrada/Salida (E/S):

1. Leer el **bloque de datos del directorio padre** actual. Un directorio no es más que un archivo que contiene una tabla con pares de `(nombre_de_archivo, número_de_i-nodo)`. El sistema busca aquí el nombre del siguiente componente de la ruta.
2. Leer el **i-nodo** correspondiente a ese número recién encontrado, trayéndolo desde el disco hacia la memoria.

Cada una de estas lecturas constituye una operación de disco independiente.

Análisis paso a paso

La ruta a resolver es: `/usr/ast/courses/os/handout.t`

El i-nodo de `/` (raíz) ya está cargado en la memoria RAM, por lo que su obtención inicial genera cero operaciones de disco. A partir de ahí, el sistema debe descender por 5 niveles:

Paso	`Nivel`	Operación de I/O en Disco	Lecturas
1	`usr`	Leer bloque de datos de `/` (buscando üsr")	1
2	`usr`	Leer i-nodo de `usr`	1
3	`ast`	Leer bloque de datos de `usr` (buscando .ªst")	1
4	`ast`	Leer i-nodo de `ast`	1
5	`courses`	Leer bloque de datos de `ast` (buscando çourses")	1
6	`courses`	Leer i-nodo de `courses`	1
7	`os`	Leer bloque de datos de `courses` (buscando .ºs")	1
8	`os`	Leer i-nodo de `os`	1
9	`handout.t`	Leer bloque de datos de `os` (buscando "handout.t")	1
10	`handout.t`	Leer i-nodo de `handout.t` (¡Archivo localizado!)	1
		Total	**10**

Fórmula general

Para una ruta absoluta con n componentes (sin contar la raíz), asumiendo que el i-nodo de la raíz ya reside en memoria y que cada directorio ocupa un solo bloque:

$$\text{Operaciones de disco} = 2 \times n$$

En este caso particular de la ruta propuesta, tenemos $n = 5$ componentes, por lo tanto:

$$\text{Operaciones de disco} = 2 \times 5 = \mathbf{10}$$

Conclusión. Se requieren estrictamente **10 operaciones de disco** para obtener el i-nodo final de `handout.t`. Este ejercicio teórico ilustra el altísimo costo computacional de la resolución de rutas en UNIX y motiva la existencia arquitectónica de las cachés de i-nodos (*inode cache*) y de entradas de directorio (*dentry cache*) en el kernel. En sistemas reales modernos, tras un primer acceso, estas estructuras se mantienen en RAM,

reduciendo los accesos a disco repetidos de 10 a 0.

Referencias

[1] Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson Education.

[2] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). John Wiley & Sons.

[3] Ritchie, D. M., & Thompson, K. (1974). The UNIX time-sharing system. *Communications of the ACM*, 17(7), 365–375. https://doi.org/10.1145/361011.361061